

THE SAFETY NET

Handling Mistakes in Python

PYTHON PROGRAMMING | GRADE 4

PLANRS Academy

Lesson 1: The Punctured Rickshaw

Imagine This...

Imagine you're going to school in a colorful rickshaw, excited for the day, and suddenly... PUNCTURE!



Visual: *A bright rickshaw with a flat tire on a busy Indian street.*

The tire goes flat and your ride stops! Uh-oh! What happens now?

Unexpected problems like this can happen anytime—even to our programs! In coding, we call these surprise problems **Exceptions**.

The Surprise Problem

What is an Exception?

An Exception is an unexpected problem that stops our program from working properly.

 **Visual:** A smiley warning sign representing a manageable error.

Remember This!

- Exception = Surprise problem.
- It can make the program stop suddenly.
- But don't worry, we can handle it!

Just like a rickshaw puncture stops your ride to school, an exception stops your code!

The Test Area: 'try'

What is 'try'?

The try block is like testing if you have enough milk at home before you start making Chai.



Visual: *A child in a kitchen pouring milk to prepare tea.*

We put code that might cause a problem inside 'try' to test it safely!

```
try:  
    make_Tea() # We try this first!
```

The Backup Plan: 'except'

What is 'except'?

If our first plan doesn't work, we need a Backup Plan. That is what except is for!



Visual: *A child running to a local Kirana store.*



The Kirana Analogy:

If the milk is finished at home (the error), our backup plan is to go to the Kirana store to buy more!

```
except:  
    go_to_kirana_store()
```

Friendly Calculators

ZeroDivisionError

Trying to divide by zero is like trying to share sweets when there are zero friends—it just doesn't work!



Visual: A child looking confused at a calculator with a warning.

```
try:
    result = number / 0
except ZeroDivisionError:
    print("Oopsie! You can't divide by zero.")
```

With try and except, our calculator stays friendly instead of crashing!

Real-World Safety Nets

Graceful Failure

Graceful failure means saying "Oopsie!" nicely instead of stopping suddenly.

- **ATM Example:** If an ATM runs out of cash, it shows a "No cash" message instead of switching off!
- **Power Cut Example:** When the power goes out, the Inverter switches on to save the day!



Visual: An ATM showing a friendly "Sorry, no cash available" message.

Remember: Errors are just chances to learn! Keep practicing your safety nets!

Page 7 | PLANRS Academy

Student Practice Workbook (Grade 4)

Test your debugging superpowers! Answer these questions based on what you learned about safety nets and handling exceptions.

1. What real-world vehicle emergency is used in this lesson to explain an unexpected program stop?

2. What is the special word in coding used to describe a "surprise problem" that stops your program?

3. True or False: If a program hits an exception, it is impossible for us to handle it or fix it.

4. What household task does the lesson mention to explain how the `try` block safely checks conditions first?

5. Where do we put the lines of code that we suspect might cause a surprise problem or error?

6. In Python, what block serves as our absolute "Backup Plan" if the code inside `try` fails?

7. According to the Kirana store analogy, if running out of milk is the error, what action represents the `except` block?

8. Why can't a calculator divide a number by zero? What example does the book use to explain this?

9. Write the specific name of the exact error Python shows when code tries to divide a number by zero.

10. What message does the program print out to keep the calculator friendly if a child divides by zero?

11. What phrase means a program says "Oopsie!" nicely and handles errors smoothly instead of crashing completely?

12. Imagine an ATM runs completely out of cash money. What does a graceful failure look like for that ATM?

13. What electrical safety appliance at home acts as a backup plan when there is a sudden power cut?

14. When writing code, what should we remember about making mistakes or running into errors?

15. Draw a line connecting the block to its function: `try` checks for errors / `except` runs the backup code.
